

Отчёт по практической работе

Разработка веб-форума

Этапы 1–3

Хворов Андрей Максимович, гр. 327

2026

Содержание

1	Введение	2
2	Этап 1. Проектирование базы данных	3
2.1	Состав базы данных	3
2.2	Поля таблиц	3
2.3	Связи между таблицами	5
3	Этап 2. ORM-слой	6
3.1	Структура проекта	6
3.2	Конфигурация ORM	6
3.3	Модели	6
3.4	DAO-классы	6
3.5	Модульные тесты	8
4	Этап 3. Веб-интерфейс	9
4.1	Структура проекта	9
4.2	Зависимости	10
4.3	Middleware	10
4.4	HTTP-маршруты	11
4.5	EJS-шаблоны	11
4.6	Сценарии использования	12
4.7	Системные тесты	12
4.8	Результаты тестирования	13
5	Запуск проекта	14
5.1	Установка зависимостей	14
5.2	Запуск тестов	14
5.3	Запуск сервера	14
6	Заключение	15

1 Введение

В рамках практической работы разработан полнофункциональный веб-форум. Работа состоит из трёх этапов:

1. **Этап 1.** Проектирование реляционной базы данных.
2. **Этап 2.** Реализация ORM-слоя (модели, DAO-классы, модульные тесты).
3. **Этап 3.** Разработка веб-интерфейса (маршруты, шаблоны, системные тесты).

Исходное задание предполагает стек Java / Hibernate / Spring / JSP / Selenium. В данной работе применяется функционально эквивалентный стек на платформе Node.js.

Таблица 1: Соответствие технологических стеков

Оригинальный стек	Используемый стек
Java	JavaScript (Node.js)
Hibernate ORM	Sequelize ORM 6
Spring MVC	Express.js 4
JSP-страницы	EJS-шаблоны
Spring Security	express-session + bcryptjs
Selenium / HTTPUnit	supertest + Jest 29
Ant build.xml	npm scripts (package.json)
PostgreSQL (prod)	PostgreSQL / SQLite :memory: (test)

2 Этап 1. Проектирование базы данных

2.1 Состав базы данных

База данных состоит из пяти таблиц. Все внешние ключи определены с ограничением ON DELETE CASCADE.

Таблица 2: Таблицы базы данных

Таблица	Назначение	Первичный ключ
user	Пользователи форума	user_id
section	Разделы форума	section_id
topic	Темы в разделах	topic_id
message	Сообщения в темах	message_id
attachment	Вложения к сообщениям	attachment_id

2.2 Поля таблиц

Таблица 3: Поля таблицы user

Поле	Тип	Ограничения	По умолч.
user_id	SERIAL	PRIMARY KEY	—
login	VARCHAR(64)	NOT NULL, UNIQUE	—
password	VARCHAR(255)	NOT NULL	—
registered_at	TIMESTAMPTZ	NOT NULL	NOW()
role	VARCHAR(16)	CHECK IN ('user', 'moderator')	'user'
is_blocked	BOOLEAN	NOT NULL	FALSE

Таблица 4: Поля таблицы section

Поле	Тип	Ограничения	По умолч.
section_id	SERIAL	PRIMARY KEY	—
title	VARCHAR(128)	NOT NULL	—
description	TEXT	—	—
user_id	INTEGER	FK → user, NOT NULL	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()

Таблица 5: Поля таблицы `topic`

Поле	Тип	Ограничения	По умолч.
<code>topic_id</code>	SERIAL	PRIMARY KEY	—
<code>section_id</code>	INTEGER	FK → section, NOT NULL	—
<code>user_id</code>	INTEGER	FK → user, NOT NULL	—
<code>title</code>	VARCHAR(255)	NOT NULL	—
<code>created_at</code>	TIMESTAMPTZ	NOT NULL	NOW()

Таблица 6: Поля таблицы `message`

Поле	Тип	Ограничения	По умолч.
<code>message_id</code>	SERIAL	PRIMARY KEY	—
<code>topic_id</code>	INTEGER	FK → topic, NOT NULL	—
<code>user_id</code>	INTEGER	FK → user, NOT NULL	—
<code>title</code>	VARCHAR(255)	—	—
<code>body</code>	TEXT	NOT NULL	—
<code>posted_at</code>	TIMESTAMPTZ	NOT NULL	NOW()

Таблица 7: Поля таблицы `attachment`

Поле	Тип	Ограничения	По умолч.
<code>attachment_id</code>	SERIAL	PRIMARY KEY	—
<code>message_id</code>	INTEGER	FK → message, NOT NULL	—
<code>filename</code>	VARCHAR(255)	NOT NULL	—
<code>filepath</code>	VARCHAR(512)	NOT NULL	—
<code>filesize</code>	INTEGER	NOT NULL	—
<code>media_type</code>	VARCHAR(128)	—	—
<code>uploaded_at</code>	TIMESTAMPTZ	NOT NULL	NOW()

2.3 Связи между таблицами

Таблица 8: Связи между таблицами

Родительская	Дочерняя	Тип связи
user	section	один ко многим
user	topic	один ко многим
user	message	один ко многим
section	topic	один ко многим
topic	message	один ко многим
message	attachment	один ко многим

3 Этап 2. ORM-слой

3.1 Структура проекта

```
wtpractice_Khvorov/  
  package.json  
  jest.config.js  
  src/  
    config/  
      database.js  
    models/  
      index.js  
      User.js  
      Section.js  
      Topic.js  
      Message.js  
      Attachment.js  
    dao/  
      UserDao.js  
      SectionDao.js  
      TopicDao.js  
      MessageDao.js  
      AttachmentDao.js  
    tests/  
      setup.js  
      UserDao.test.js  
      SectionDao.test.js  
      TopicDao.test.js  
      MessageDao.test.js  
      AttachmentDao.test.js
```

Рис. 1: Структура проекта (этап 2)

3.2 Конфигурация ORM

Конфигурация определена в `src/config/database.js` и выбирается по переменной окружения `NODE_ENV`. В режиме `development` используется PostgreSQL, в режиме `test` — SQLite в оперативной памяти (`:memory:`). Опция `underscored: true` обеспечивает автоматическое преобразование camelCase-имён полей в snake_case-имена столбцов.

3.3 Модели

Каждая модель определяется через `sequelize.define()` с явным указанием `tableName`. Ассоциации задаются в `src/models/index.js` через `hasMany` / `belongsTo` с псевдонимами: `creator` (автор раздела), `author` (автор темы или сообщения), `attachments` (вложения сообщения). Для всех связей указан `onDelete: 'CASCADE'`.

3.4 DAO-классы

Каждый DAO-класс инкапсулирует логику доступа к данным одной сущности и экспортируется как синглтон (`module.exports = new SomeDao()`). Все методы асин-

хронные.

Таблица 9: Методы UserDao

Метод	Описание
<code>createUser(login, password, role)</code>	Создание пользователя. Роль по умолчанию <code>user</code> .
<code>findByLogin(login)</code>	Поиск по логину, возвращает объект или <code>null</code> .
<code>findById(id)</code>	Поиск по первичному ключу.
<code>findAll(filters)</code>	Список пользователей с фильтрами по роли и статусу блокировки.
<code>updateRole(userId, newRole)</code>	Смена роли.
<code>setBlocked(userId, isBlocked)</code>	Блокировка / разблокировка.
<code>updatePassword(userId, newPassword)</code>	Смена пароля.
<code>getUserStats(userId)</code>	Статистика: число сообщений и тем пользователя.

Таблица 10: Методы SectionDao

Метод	Описание
<code>createSection(title, desc, userId)</code>	Создание раздела.
<code>findAll()</code>	Все разделы с данными создателя, числом тем и последним сообщением.
<code>findById(sectionId)</code>	Поиск по РК с данными создателя.
<code>findByTitle(title)</code>	Поиск по названию.
<code>deleteSection(sectionId)</code>	Каскадное удаление раздела.

Таблица 11: Методы TopicDao

Метод	Описание
<code>createTopic(sectionId, userId, title)</code>	Создание темы.
<code>findBySectionId(sectionId, options)</code>	Темы раздела с пагинацией и сортировкой.
<code>findById(topicId)</code>	Поиск по РК с данными автора.
<code>deleteTopic(topicId)</code>	Каскадное удаление темы.
<code>getTopicsByUser(userId)</code>	Все темы пользователя.
<code>countBySectionId(sectionId)</code>	Количество тем в разделе.

Таблица 12: Методы MessageDao

Метод	Описание
<code>createMessage(topicId, userId, body, title)</code>	Создание сообщения.
<code>findByTopicId(topicId, options)</code>	Сообщения с данными автора и вложениями, пагинация.
<code>deleteMessage(messageId)</code>	Каскадное удаление сообщения.
<code>countByUserId(userId)</code>	Число сообщений пользователя.
<code>getLastMessageInSection(sectionId)</code>	Последнее сообщение в разделе.
<code>getActivityStats(dateFrom, dateTo, ...)</code>	Статистика активности по дням.

Таблица 13: Методы AttachmentDao

Метод	Описание
<code>createAttachment(msgId, name, path, size, type)</code>	Создание вложения.
<code>findByMessageId(messageId)</code>	Все вложения сообщения.
<code>findById(attachmentId)</code>	Поиск по РК.
<code>deleteByMessageId(messageId)</code>	Удаление всех вложений сообщения.

3.5 Модульные тесты

Тесты написаны на Jest. Перед каждым тестом выполняется `sync({force: true})` и заполнение фиксированными данными: 4 пользователя, 3 раздела, 4 темы, 5 сообщений, 3 вложения. Для каждого метода DAO протестированы все варианты поведения (успешный результат, пустой результат, ошибка валидации).

Таблица 14: Результаты модульного тестирования

Файл	Тестов	Stmts	Branch	Funcs
UserDao.js	24	100%	100%	100%
SectionDao.js	10	100%	50%	100%
TopicDao.js	12	100%	100%	100%
MessageDao.js	18	100%	100%	100%
AttachmentDao.js	8	100%	100%	100%
Итого	72	100%	93,87%	100%

Непокрытые ветви (6,13%) относятся к инфраструктурному коду: выбор конфигурации в `database.js` и проверка диалекта в `setup.js`.

4 Этап 3. Веб-интерфейс

4.1 Структура проекта

```
wtpractice_Khvorov/  
  package.json  
  src/  
    app.js  
    server.js  
    middleware/  
      auth.js  
      flash.js  
      upload.js  
    routes/  
      auth.js  
      sections.js  
      topics.js  
      users.js  
      admin.js  
  views/  
    partials/  
      header.ejs  
      footer.ejs  
    index.ejs  
    register.ejs  
    login.ejs  
    section.ejs  
    topic.ejs  
    user.ejs  
    admin.ejs  
    error.ejs  
  tests/  
    system/  
      setup.js  
      agent.js  
      auth.test.js  
      sections.test.js  
      topics.test.js  
      users.test.js  
      admin.test.js
```

Рис. 2: Структура проекта (этап 3)

4.2 Зависимости

Таблица 15: Зависимости, добавленные на этапе 3

Пакет	Назначение
express	HTTP-сервер и маршрутизация
ejs	Шаблонизатор (аналог JSP)
express-session	Сессии на стороне сервера
bcryptjs	Хэширование паролей (cost=10)
multer	Загрузка файлов на диск
supertest	HTTP-тесты (аналог HTTPUnit / Selenium)

4.3 Middleware

`auth.js` содержит три функции. `injectUser` на каждом запросе подгружает пользователя из БД и принудительно завершает сессию заблокированного пользователя. `requireAuth` перенаправляет неавторизованного пользователя на `/login`. `requireModerator` возвращает статус 403 при попытке доступа пользователя с ролью `user`.

`upload.js` настраивает `multer` с дисковым хранилищем в директории `/uploads` и ограничением размера 10 МБ.

`flash.js` реализует одноразовые сообщения через сессию.

4.4 HTTP-маршруты

Таблица 16: Маршруты приложения

Метод и путь	Доступ	Описание
GET /	Все	Список разделов
GET /section/:id	Все	Темы в разделе
GET /topic/:id	Все	Сообщения в теме
GET /user/:id	Все	Профиль пользователя
GET /register	Гость	Форма регистрации
POST /register	Гость	Регистрация
GET /login	Гость	Форма входа
POST /login	Гость	Вход
POST /logout	Авторизован	Выход
POST /section/:id/topic	Авторизован	Создание темы
POST /topic/:id/message	Авторизован	Создание сообщения
POST /user/:id/password	Владелец	Смена пароля
GET /admin	Модератор	Панель управления
GET /admin/stats	Модератор	Статистика активности
POST /admin/section	Модератор	Создание раздела
POST /admin/section/:id/delete	Модератор	Удаление раздела
POST /admin/user/:id/block	Модератор	Блокировка пользователя
POST /admin/user/:id/unblock	Модератор	Разблокировка
POST /admin/user/:id/role	Модератор	Изменение роли
POST /topic/:id/delete	Модератор	Удаление темы
POST /message/:id/delete	Модератор	Удаление сообщения

4.5 EJS-шаблоны

Все страницы используют общую шапку и футтер через `include('partials/header')` и `include('partials/footer')`. Переменная `res.locals.currentUser` доступна во всех шаблонах. Формы, выполняющие деструктивные операции, защищены диалогом подтверждения через `onsubmit="return confirm(...)"`.

4.6 Сценарии использования

Таблица 17: Реализованные сценарии использования

Сценарий	Доступ	Страница
Регистрация	Гость	/register
Авторизация / Выход	Гость	/login
Просмотр разделов	Все	/
Просмотр тем в разделе	Все	/section/:id
Просмотр сообщений в теме	Все	/topic/:id
Создание темы	Авторизован	/section/:id
Создание сообщения	Авторизован	/topic/:id
Просмотр профиля	Все	/user/:id
Смена пароля	Владелец профиля	/user/:id
Создание раздела	Модератор	/admin
Удаление раздела	Модератор	/admin
Удаление темы / сообщения	Модератор	/section/:id, /topic/:id
Блокировка пользователя	Модератор	/user/:id
Назначение модератора	Модератор	/user/:id
Статистика активности	Модератор	/admin/stats

4.7 Системные тесты

Тесты написаны на supertest + Jest. Агент `request.agent(app)` сохраняет cookie между запросами. Перед каждым тестом выполняется `systemSeed()`: пересоздание схемы БД, заполнение фиксированными данными и перезапись паролей реальными bcrypt-хэшами (cost=4 для ускорения). Для каждой операции с несколькими возможными ошибками написан отдельный тест на каждое сообщение.

Таблица 18: Системные тесты по файлам

Файл	Тестов	Покрытые сценарии
auth.test.js	13	Регистрация (6 вариантов), вход (5 вариантов), выход, принудительный выход заблокированного
sections.test.js	12	Список разделов (5), темы в разделе: просмотр, сортировка, пагинация, 404 (7)
topics.test.js	20	Просмотр темы (8), создание темы (6), создание сообщения (5), удаление темы (3), удаление сообщения (3)
users.test.js	16	Просмотр профиля с разными ролями (9), смена пароля: 5 ошибок, успех, 403, redirect (8)
admin.test.js	30	Создание раздела (6), удаление раздела (5), блокировка (8), назначение роли (6), статистика (9)
Итого	91	

4.8 Результаты тестирования

Все тесты проходят успешно (178 тестов обоих этапов).

```
NODE_ENV=test npm test
```

```
Test Suites: 10 passed, 10 total
```

```
Tests:      178 passed, 178 total
```

```
Time:       7 s
```

5 Запуск проекта

5.1 Установка зависимостей

```
npm install
```

5.2 Запуск тестов

```
NODE_ENV=test npm test
```

5.3 Запуск сервера

Для работы в режиме разработки необходим PostgreSQL. Параметры подключения задаются через переменные окружения в `src/config/database.js`.

```
npm start
```

После запуска приложение доступно по адресу `http://localhost:3000`.

6 Заключение

В ходе трёх этапов практической работы разработан полнофункциональный веб-форум.

На **первом этапе** спроектирована реляционная база данных из 5 таблиц с ограничениями целостности и каскадным удалением.

На **втором этапе** реализован ORM-слой: 5 моделей Sequelize и 5 DAO-классов. Написаны 72 модульных теста с покрытием 100% по строкам и функциям.

На **третьем этапе** разработан веб-интерфейс: 21 HTTP-маршрут, 9 EJS-шаблонов, 3 middleware-функции, загрузка файлов. Написан 91 системный тест, покрывающие все сценарии использования и все варианты сообщений об ошибках.

Использование Node.js / Sequelize / Express обеспечивает функциональную эквивалентность исходному заданию на Java / Hibernate / Spring при сохранении тех же архитектурных паттернов: entity-модели, DAO-слой, модульные и системные тесты с изолированным окружением.